06-02 동시성 처리 분석

개요

Rain BitTorrent 클라이언트는 수백 개의 토렌트와 수천 개의 피어 연결을 동시에 처리할 수 있도록 설계된 고도로 동시적인 시스템입니다. 이 문서는 Rain에서 사용하는 고루틴 관리 전략, 채널 기반 메시지 전달, 동기화 패턴, 그리고 데드락 방지 기법을 상세히 분석합니다.

고루틴 관리 전략

계층적 고루틴 구조

Rain은 계층적 고루틴 구조를 사용하여 시스템의 복잡성을 관리합니다:

```
Session (최상위)
├── DHT 처리 고루틴
├── 통계 업데이트 고루틴
├── RPC 서버 고루틴
└── Torrent 고루틴들
  ├── 토렌트 메인 루프
  ├── Allocator 고루틴
  ├── Verifier 고루틴
  ├── Announcer 고루틴들
  └── Peer 고루틴들
    ├── 피어 메시지 처리
```

```
├── 인커밍/아웃고잉 핸드셰이커
└── PEX 고루틴
```

토렌트 메인 루프

각 토렌트는 단일 고루틴에서 실행되는 메인 이벤트 루프를 가집니다:

```go
func (t *torrent) run() {
t.seedDurationTicker = time.NewTicker(time.Second)
defer  t.seedDurationTicker.Stop()

t.unchokeTicker = time.NewTicker(10 * time.Second)
defer  t.unchokeTicker.Stop()

for {
select {
case <-t.closeC:
t.close()
close(t.doneC)
return
case <-t.startCommandC:
t.start()
case <-t.stopCommandC:
t.stop(nil)
```

```
case <-t.announceCommandC:
t.setNeedMorePeers(true)
case <-t.verifyCommandC:
t.handleVerifyCommand()
// ... 30개 이상의 채널 케이스들
case  pm := <-t.pieceMessagesC.ReceiveC():
t.handlePieceMessage(pm)
case  pm := <-t.messages:
t.handlePeerMessage(pm)
}
}
}
```

이 패턴의 장점:

- **단일 스레드 모델**: 토렌트 상태 변경이 순차적으로 처리되어 경쟁 조건을 방지
- **이벤트 기반**: 각 이벤트가 명확히 정의된 핸들러로 처리
- **확장성**: 새로운 이벤트 타입을 쉽게 추가 가능

피어별 고루틴 관리

각 피어 연결은 독립적인 고루틴에서 처리됩니다:

```
func (p *Peer) Run(messages  chan Message, pieces  chan PieceMessage,
snubbed, disconnect  chan *Peer) {
defer  close(p.doneC)
```

```
go  p.Conn.Run() // 별도 고루틴에서 네트워크 I/O 처리


for {
select {
case  pm, ok := <-p.Conn.Messages():
if !ok {
select {
case  disconnect <- p:
case <-p.closeC:
}
return
}
// 메시지 타입별 처리
if  m, ok := pm.(peerreader.Piece);  ok {
p.downloadSpeed.Mark(int64(len(m.Buffer.Data)))
select {
case  pieces <- PieceMessage{Peer: p, Piece: m}:
case <-p.closeC:
return
}
} else {
select {
case  messages <- Message{Peer: p, Message: pm}:
```

```
case <-p.closeC:
return
}
}
case <-p.snubTimer.C:
select {
case  snubbed <- p:
case <-p.closeC:
return
}
case <-p.closeC:
return
}
}
}
```

고루틴 생명주기 관리

Rain은 체계적인 고루틴 생명주기 관리를 구현합니다:

```
// 피어 연결 시작
func (t *torrent) startPeer(conn net.Conn, source peersource.Source,
peers  map[*peer.Peer]struct{}, peerID [20]byte,
extensions [8]byte, cipher mse.CryptoMethod) {
```

```
pe := peer.New(conn, source, peerID, extensions, cipher,
t.session.config.PieceReadTimeout,
t.session.config.RequestTimeout,
t.session.config.MaxRequestsIn,
t.session.bucketDownload, t.session.bucketUpload)
t.peers[pe] = struct{}{}
peers[pe] = struct{}{}
// 피어 고루틴 시작
go pe.Run(t.messages, t.pieceMessagesC.SendC(),
t.peerSnubbedC, t.peerDisconnectedC)
t.session.metrics.Peers.Inc(1)
t.sendFirstMessage(pe)
}


// 세션 종료 시 모든 토렌트 고루틴 대기
func (s *Session) Close() error {
close(s.closeC)


var wg sync.WaitGroup
s.mTorrents.Lock()
wg.Add(len(s.torrents))
for _, t := range s.torrents {
go func(t *Torrent) {
```

```
t.torrent.Close() // 각 토렌트의 정리 작업
wg.Done()
}(t)
}
wg.Wait() // 모든 토렌트가 종료될 때까지 대기
s.mTorrents.Unlock()
return s.db.Close()
}
```

채널을 이용한 메시지 전달

SuspendChan 패키지

Rain은 채널의 일시 중단 기능을 제공하는 `SuspendChan` 을 구현합니다:

```
type Chan[T any] struct {
ch chan T
suspended bool
}

func (c *Chan[T]) ReceiveC() chan T {
if c.suspended {
return nil // nil 채널은 항상 블록됨
}
```

```
return c.ch
}

func (c *Chan[T]) Suspend() {
c.suspended = true
}

func (c *Chan[T]) Resume() {
c.suspended = false
}
```

이 패턴의 활용:

- **백프레셔 제어**: 시스템 과부하 시 특정 메시지 흐름을 일시 중단
- **우선순위 관리**: 중요한 메시지를 우선 처리하기 위해 다른 채널을 중단
- **동적 흐름 제어**: 런타임에 메시지 흐름을 조절

메시지 타입별 채널 분리

Rain은 메시지 타입에 따라 채널을 분리하여 성능을 최적화합니다:

```
// 토렌트 구조체의 채널들
type torrent struct {
// 명령 채널들
startCommandC chan struct{}
stopCommandC chan struct{}
```

```
announceCommandC chan struct{}
verifyCommandC chan struct{}
// 데이터 채널들
messages chan peer.Message
pieceMessagesC *suspendchan.Chan[peer.PieceMessage]
// 결과 채널들
allocatorResultC chan *allocator.Allocator
verifierResultC chan *verifier.Verifier
pieceWriterResultC chan *piecewriter.PieceWriter
// 이벤트 채널들
peerDisconnectedC chan *peer.Peer
peerSnubbedC chan *peer.Peer
incomingConnC chan net.Conn
}
```

채널 버퍼링 전략

Rain은 채널 타입에 따라 다른 버퍼링 전략을 사용합니다:

1. **무버퍼 채널**: 동기화가 필요한 명령 채널
2. **작은 버퍼**: 이벤트 채널 (1-10)
3. **큰 버퍼**: 고처리량 데이터 채널 (100-1000)

```
// suspendchan 생성 시 버퍼 크기 지정
func New[T any](buflen int) *Chan[T] {
```

```
return &Chan[T]{
ch: make(chan T, buflen),
}
}
```

세마포어를 이용한 동시성 제어

커스텀 세마포어 구현

Rain은 고성능 세마포어를 직접 구현합니다:

```
type Semaphore struct {
c  chan token
waiting  int32  // 대기 중인 고루틴 수
active  int32  // 활성 고루틴 수
}

func (s *Semaphore) Wait() {
atomic.AddInt32(&s.waiting, 1)
s.c <- token{} // 블로킹 대기
atomic.AddInt32(&s.waiting, -1)
atomic.AddInt32(&s.active, 1)
}
```

```go
func (s *Semaphore) Signal() {
<-s.c
atomic.AddInt32(&s.active, -1)
}
```

세마포어 활용 사례

1. **파일 I/O 제한**:

```go
// 세션 생성 시
s.semWrite = semaphore.New(int(cfg.ParallelWrites))

// 피스 쓰기 시
func (w *PieceWriter) Run(resultC  chan *PieceWriter, closeC  chan  struct{},
writesPerSecond, writeBytesPerSecond metrics.Meter,
sem *semaphore.Semaphore) {
w.HashOK = w.Piece.VerifyHash(w.Buffer.Data, sha1.New())
if  w.HashOK {
sem.Wait() // 동시 쓰기 제한
_, w.Error = w.Piece.Data.Write(w.Buffer.Data)
sem.Signal()
}
}
```

2. **캐시 로드 제한**:

```
func (c *Cache) getValue(i *item, loader Loader) ([]byte, error) {
i.Lock()
defer i.Unlock()

if i.loaded {
return i.value, nil
}

c.sem.Wait() // 동시 로드 제한
i.value, i.err = loader()
c.sem.Signal()
return c.handleNewItem(i)
}
```

세마포어 모니터링

Rain은 세마포어 상태를 실시간으로 모니터링합니다:

```
func (c *Cache) LoadsActive() int {
return c.sem.Len() // 현재 활성 로드 수
}

func (c *Cache) LoadsWaiting() int {
```

```
return c.sem.Waiting() // 대기 중인 로드 수
}
```

뮤텍스와 동기화 패턴

읽기-쓰기 뮤텍스 활용

Rain은 읽기 작업이 많은 데이터 구조에 `sync.RWMutex` 를 적극 활용합니다:

```
type Session struct {
mTorrents sync.RWMutex
torrents map[string]*Torrent
torrentsByInfoHash map[dht.InfoHash][]*Torrent
mPorts sync.RWMutex
availablePorts map[int]struct{}
mBlocklist sync.RWMutex
blocklist *blocklist.Blocklist
}

// 읽기 작업 (빈번함)
func (s *Session) GetTorrent(id string) *Torrent {
s.mTorrents.RLock()
defer s.mTorrents.RUnlock()
return s.torrents[id]
```

```go
}

// 쓰기 작업 (드물음)
func (s *Session) removeTorrentFromClient(id string) (*Torrent, error) {
s.mTorrents.Lock()
defer s.mTorrents.Unlock()
t, ok := s.torrents[id]
if !ok {
return nil, nil
}
delete(s.torrents, id)
// ... 추가 정리 작업
return t, nil
}
```

세밀한 락 분할

Rain은 락 경합을 줄이기 위해 데이터를 논리적으로 분할하여 별도의 뮤텍스로 보호합니다:

```go
type Session struct {
// 토렌트 관련 락
mTorrents sync.RWMutex
torrents map[string]*Torrent
// 포트 관련 락 (별도)
```

```
mPorts sync.RWMutex
availablePorts  map[int]struct{}
// 블랙리스트 관련 락 (별도)
mBlocklist sync.RWMutex
blocklist *blocklist.Blocklist
// DHT 피어 요청 관련 락 (별도)
mPeerRequests sync.Mutex
dhtPeerRequests  map[*torrent]struct{}
}
```

아토믹 연산 활용

성능이 중요한 카운터에는 `sync/atomic` 패키지를 사용합니다:

```
type Semaphore struct {
c  chan token
waiting  int32  // atomic으로 접근
active  int32  // atomic으로 접근
}

func (s *Semaphore) Waiting() int {
return  int(atomic.LoadInt32(&s.waiting))
}
```

```go
func (s *Semaphore) Len() int {
return int(atomic.LoadInt32(&s.active))
}
```

데드락 방지 기법

락 순서 규칙

Rain은 일관된 락 획득 순서를 유지하여 데드락을 방지합니다:

```go
// 항상 mTorrents -> DHT 순서로 락 획득
func (s *Session) removeTorrentFromClient(id string) (*Torrent, error) {
s.mTorrents.Lock()
t, ok := s.torrents[id]
if !ok {
s.mTorrents.Unlock()
return nil, nil
}
delete(s.torrents, id)
// DHT 작업 전에 mTorrents 락을 먼저 해제
// DHT.RemoveInfoHash가 내부적으로 다른 락을 요구할 수 있음
s.mTorrents.Unlock()
if s.config.DHTEnabled && len(s.torrentsByInfoHash[ih]) == 0 {
s.dht.RemoveInfoHash(string(ih)) // 별도 고루틴에서 처리
```

```
}
return t, nil
}
```

타임아웃 기반 데드락 방지

채널 작업에서 타임아웃을 사용하여 무한 대기를 방지합니다:

```
func (t *torrent) resolveAndAddPeer(host string, port int) {
ctx, cancel := context.WithCancel(context.Background())
done := make(chan struct{})
go func() {
select {
case <-t.closeC: // 토렌트 종료 시
case <-done: // 정상 완료 시
}
cancel() // 컨텍스트 취소로 타임아웃 처리
}()
ip, err := resolver.ResolveIPv4(ctx, t.session.config.DNSResolveTimeout, host)
if err != nil {
return
}
// ... 피어 추가 로직
```

```
}
```

비블로킹 채널 패턴

중요한 채널 작업에서 비블로킹 패턴을 사용합니다:

```
func (a *Allocator) sendProgress(progressC chan Progress, size int64) {
select {
case progressC <- Progress{AllocatedSize: size}:
// 성공적으로 전송
case <-a.closeC:
return // 할당자가 종료되면 즉시 반환
}
}

func (w *PieceWriter) Run(resultC chan *PieceWriter, closeC chan struct{},
writesPerSecond, writeBytesPerSecond metrics.Meter,
sem *semaphore.Semaphore) {
// ... 피스 쓰기 로직
select {
case resultC <- w:
// 결과 전송 성공
case <-closeC:
// 종료 신호 시 즉시 반환
```

```
}

}
```

고루틴 리크 방지

모든 고루틴이 적절히 종료되도록 보장합니다:

```
func (p *Peer) Close() {

p.snubTimer.Stop()

if  p.PEX != nil {

p.PEX.close()

}

close(p.closeC) // 종료 신호 전송

p.Conn.Close() // 네트워크 연결 종료

p.downloadSpeed.Stop()

p.uploadSpeed.Stop()

<-p.doneC  // 고루틴 종료 대기

}


func (p *Peer) Run(messages  chan Message, pieces  chan PieceMessage,

snubbed, disconnect  chan *Peer) {

defer  close(p.doneC) // 함수 종료 시 완료 신호

go  p.Conn.Run()

for {
```

```
select {
// ... 다양한 케이스들
case <-p.closeC:
return  // 종료 신호 시 즉시 반환
}
}
}
```

성능 최적화 패턴

고루틴 풀링

Rain은 고루틴을 동적으로 생성하지만, 일부 작업에서는 암묵적 풀링을 사용합니다:

```
// 피어 연결 시 고루틴 재사용
func (t *torrent) dialAddresses() {
for  peersConnected() < t.session.config.MaxPeerDial {
addr, src := t.addrList.Pop()
if  addr == nil {
return
}
h := outgoinghandshaker.New(addr, src)
t.outgoingHandshakers[h] = struct{}{}
// 각 핸드셰이커마다 새 고루틴 생성
```

```
go h.Run(/* ... parameters ... */)
}
}
```

배치 처리

여러 작업을 배치로 처리하여 동기화 오버헤드를 줄입니다:

```
func (s *Session) StartAll() error {
// 데이터베이스 업데이트를 배치로 처리
err := s.db.Update(func(tx *bbolt.Tx) error {
tb := tx.Bucket(torrentsBucket)
s.mTorrents.RLock()
for _, t := range s.torrents {
b := tb.Bucket([]byte(t.torrent.id))
_ = b.Put([]byte("started"), []byte("true"))
}
defer s.mTorrents.RUnlock()
return nil
})
// 모든 토렌트 시작
for _, t := range s.torrents {
t.torrent.Start()
}
```

```
return nil
}
```

결론

Rain BitTorrent 클라이언트의 동시성 처리는 다음과 같은 핵심 원칙을 따릅니다:

1. **계층적 고루틴 구조**: 명확한 책임 분리와 생명주기 관리
2. **채널 기반 통신**: 타입 안전하고 데드락 없는 메시지 전달
3. **세마포어 기반 제한**: 리소스 사용량 제어와 시스템 안정성
4. **세밀한 락 분할**: 경합 최소화와 성능 최적화
5. **비블로킹 패턴**: 시스템 응답성과 데드락 방지

이러한 패턴들은 고성능 네트워크 애플리케이션 개발에 있어 검증된 모범 사례들로, BitTorrent 클라이언트뿐만 아니라 다른 동시성 집약적 시스템에도 적용할 수 있습니다.

관련 문서

선행 지식

- 01-system-architecture.md - 전체 시스템 아키텍처와 계층 구조
- 02-data-flow-analysis.md - 채널 기반 메시지 전달 이해

관련 주제

- 06-01-performance-optimization-analysis.md - 성능 최적화와 동시성
- 04-01-networking-packages-analysis.md - 피어 고루틴 관리

후속 학습

- 06-03-error-handling-recovery-analysis.md - 동시성 환경에서의 에러 처리
- 03-01-torrent-lifecycle-process.md - 토렌트 고루틴 생명주기

> **네비게이션:** 이전: 성능 최적화 분석 | 목차 | 다음: 에러 처리 및 복구 분석